Word-in-Context Classification for Human-Machine Mixed Text Detection

Anya Kiewit	Isabelle	Johan Wols	Marten de Jong
Address line	S5319242	Address line	4557182
Address line	i.s.voskamp@student.rug.nl	Address line	marten.de.jong.4@student.rug.nl

Abstract

1 Introduction

2 Related Work

Maybe, Maybe not

SemEval-2024 Task 8 and Subtask C Our research is within in the framework of SemEval-2024 Task 8, which benchmarks Multigenerator, Multidomain and Multilingual Machine-Generated Text Detection. Subtask C presents the challenge of boundary detection in mixed texts. The objective is to identify the word index where a document transitions from human-written to machine-generated content. Evaluating this requires models to identify abrupt shifts in stylistic consistency and token predictability.

Stylometry and Probabilistic AI Text Detection Historically authorship attribution and text classification relied heavily on stylometric features to establish an author's unique fingerprint (Neal et al., 2017). Surface-level markers such as average word length, punctuation frequency and capitalization patterns have proven to be robust discriminators of human writing styles (Neal et al., 2017).

However with the rise of Large Language Models (LLMs), detection mechanisms have increasingly adopted probabilistic approaches. Masked Language Models (MLMs) are now frequently used to extract predictive features such as token log-probability, rank and perplexity. Recently, Xu and Sheng (2024) demonstrated this approach by utilizing MLM-based perplexity to detect AI-generated sequences (Xu and Sheng, 2024). Their findings emphasize that AI-generated text, in their case code, consistently exhibits lower and more uniform perplexity compared to the natural variance of human authorship (Xu and Sheng, 2024).

3 Data

3.1 Dataset Description

Written texts where each token separated by space is either labeled with 0 for Human written or Machine written text

Statistics of the data

The data for this study is provided by the SemEval-2024 Task 8 organizers for Subtask C (Human-Machine Mixed Text Detection). The dataset consists of documents where the first part is written by a human and the following part is generated by a LLM. The data is structured in JSON Lines (JSONL) format, where each entry contains a unique id, the raw text, and a list of labels. The label represents whether the token at the 0-based index is human-written (0) or machine-written (1).

4 Method

4.1 Pipeline

small summary of the task, what we are trying to predict

For each instance, a context window is constructed around a target word. Based on this context, a set of statistical features was extracted to capture surface-level charactetristics of the text. These were then appended to the MLM features to create a comprehensive feature vector for each instance. This feature vector is then used to train a classifier to predict whether the target word is human-written or machine-generated.

4.2 Preprocessing

Split only on (" "), lemmatization/stemming and punctuation handling kept to a minimum

The dataset was split into a training set and a test set, with an 80/20 split. The data was tokenized by

splitting on spaces, to ensure the labeling remained accurate.

4.3 Feature Engineering

Summary of the kinds of features where trying to get and what will eventually go into the models

4.3.1 Positional Features

absolute doc pos and relative doc pos

4.3.2 Statistical Features

Based on a context window, a set of statistical features was extracted to capture surface-level characteristics of the text.

These features include: (1) the average word length within the context window, (2) the number of punctuation marks, (3) a binary indicator capturing whether the target word is written in all capital letters, (4) a binary indicator whether the target word has a capital letter as its first letter, and (5) the tf-idf score of the target word. To ensure that these features reflect only meaningful textual information, padding tokens (<PAD>) were excluded from all computations.

4.3.3 MLM Features

To capture the difference between human-written and machine-generated text we used a Masked Language Model (MLM) to extract features that capture the predictability of tokens. For this we chose to use the pre-trained `roberta-large` model (Liu et al., 2019). For each context window, we mask the central target token using the tokenizer's `mask` token and pass the sequence through the model. We then extract the probability of the actual target token and its rank within the predicted probability distribution.

Our choice for `roberta-large` is motivated by its bidirectional architecture, which processes both left and right context at the same time. Recent studies have shown that bidirectional models are better at discriminating between human-written and machine-generated text than unidirectional (autoregressive) models (Huang et al., 2024). `roberta-large` has also been used in top-performing systems in the SemEval-2024 Task 8 due to its robust ability to capture deep semantic cues and structural predictability (Datta et al., 2024).

This approach is based on the hypothesis that language models generate highly predictable text. Machine-generated tokens should therefore show higher probabilities and better (lower) ranks than human-written text, which contains more variance.

Class (Split)	$\bar{p}$	$\tilde{p}$	$\bar{r}$	$\tilde{r}$
H (T)	0.367	0.151	295.4	2.0
M (T)	0.433	0.310	218.6	1.0
H (V)	0.362	0.157	288.1	2.0
M (V)	0.438	0.324	219.7	1.0

Table 1: Comparison of target token probability (p) and rank (r) derived from `roberta-large`. The symbols $\bar{x}$ and $\tilde{x}$ denote the mean and median.

MLM Feature Analysis An analysis of the extracted features (Table 1) supports our hypothesis. Across the training set, machine tokens were very predictable, with a median rank ($\tilde{r}$) of 1.0 and a median probability ($\tilde{p}$) of 0.310. In contrast, human tokens displayed lower predictability ($\tilde{p} = 0.151$). The higher mean rank for human text ($\bar{r} = 295.4$) compared to machine text ($\bar{r} = 218.6$) illustrates the "long tail" of natural human writing, showing the creative and idiosyncratic word choices that language models rarely predict.

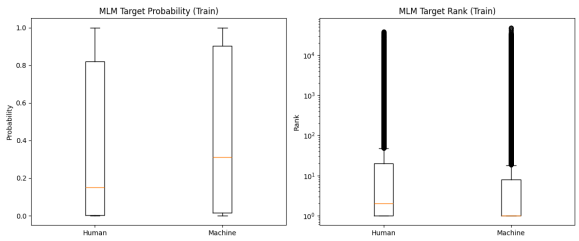

Figure 1: Distribution of MLM target token probabilities and predicted ranks for the Training set.

4.4 Models

Why we've chosen the models we chose

4.4.1 Naive Bayes

Describe Naive Bayes, and describe the baseline

4.4.2 Logistic Regression

In this project, we implemented a baseline classification model for the word-in-context task using Logistic Regression combined with manually engineered statistical features.

Given that the extracted features operate on different numerical scales, feature standardization was applied using a `StandardScaler`. This preprocessing step was integrated into a pipeline together with the classifier. The model was trained using a maximum of 1000 iterations to ensure convergence.

The overall pipeline was designed to be modular and extensible, allowing additional features to be incorporated without modifying the model architecture.

4.4.3 SVM

Describe the experiments

SVMs are maximal margin classifiers, they optimize the decision boundary by finding the hyperplane that maximizes the distance between the nearest data points (support vectors) of different classes.

As standard SVMs can be slow when dealing with very large datasets and we are analyzing hundreds of thousands of individual words, we experimented with two faster, highly scalable versions of the model: a Linear SVM (`LinearSVC`) and an SVM optimized for large-scale data (`SGDClassifier`).

For the Linear SVM we ran an automated search to fine-tune the model's parameters, to find its most accurate settings.

To measure the impact of our different features, we trained and evaluated these models twice: first using only the basic text statistics, and then adding the MLM predictability scores to see how much the context helped the model's accuracy.

SVM Performance and Feature Importance Adding MLM features improved both `LinearSVC` and `SGDClassifier`. Using basic text statistics alone, the best model (SGD) achieved 0.6947 accuracy and 0.6646 Macro F1. Incorporating MLM features boosted this to 0.7110 accuracy, 0.6786 Macro F1, and 0.83 precision for machine-generated tokens.

Figure 2 explains this improvement: while basic models rely on `tfidf_score` and `average_word_length`, the combined model shifts almost entirely to MLM features, proving them to be much stronger indicators of AI-generated text.

4.4.4 Random Forests

TODO

5 Experiments

Don't know about this yet. Maybe explicitly state why we've not chosen to do anything significant with this

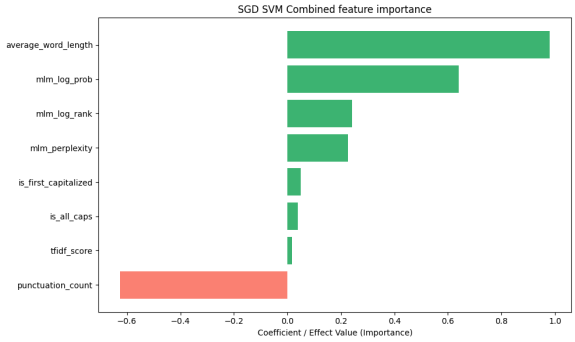

Figure 2: Feature importance for the Combined SGD SVM.

6 Results

TODO

7 Conclusion

TODO

8 Limitations

TODO

9 Future Work

Try different context window sizes. Try different models, transformer based sequence tagging

References

Ayan Datta, Aryan Chandramania, and Radhika Mamidi. 2024. Weighted layer averaging RoBERTa for black-box machine-generated text detection. In *Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024)*, pages 1623–1626. Association for Computational Linguistics.

Guanhua Huang, Yuchen Zhang, Zhe Li, Yongjian You, Mingze Wang, and Zhouwang Yang. 2024. Are AI-generated text detectors robust to adversarial perturbations? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6005–6024, Bangkok, Thailand. Association for Computational Linguistics.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Tempestt Neal, Kalaivani Sundararajan, Aneez Fatima, Yiming Yan, Yingfei Xiang, and Damon Woodard. 2017. Surveying stylometry techniques and applications. *ACM Computing Surveys (CSUR)*, 50(6):1–36.

Zhenyu Xu and Victor S. Sheng. 2024. Detecting ai-generated code assignments using perplexity of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

A Full Model Evaluation Metrics

Table 2 presents the comprehensive validation metrics across all tested models. We compared the baseline models and purely statistical models against the combined models (which include the Masked Language Model predictability features).

B Feature Importance Visualizations

The following figures illustrate the feature importances learned by each model. For linear models (Logistic Regression, SVMs), these represent the coefficients mapping to the classes. For tree-based models (Random Forests), these represent node impurities or feature impact.

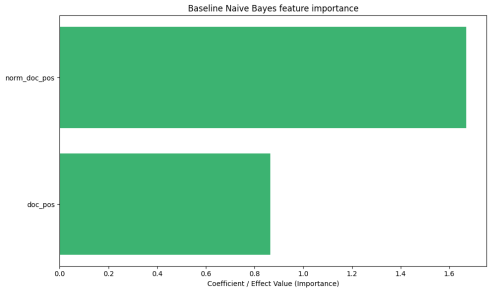

Figure 3: Feature importances for Baseline Naive Bayes.

Model	Feature Set	Validation Accuracy	Validation Macro F1	Boundary MAE
Naive Bayes	Baseline (Positional)	0.8432	0.8236	-
Naive Bayes	Statistical	0.7387	0.6094	71.54
Naive Bayes	Combined	0.7146	0.6017	71.55
Logistic Regression	Statistical	0.7576	0.6525	71.73
Logistic Regression	Combined	0.7624	0.6647	71.54
Linear SVM	Statistical	0.6887	0.6608	66.39
Linear SVM	Combined	0.7006	0.6721	66.59
SGD SVM	Statistical	0.6947	0.6646	66.71
SGD SVM	Combined	0.7110	0.6786	67.42
Random Forest	Statistical	0.7689	0.7309	62.67
Random Forest	Combined	**0.8180**	**0.7797**	**60.83**

Table 2: Comprehensive validation results for all trained models. "Combined" refers to the integration of `roberta-large` MLM features (Probability, Rank, Perplexity) with standard stylometric statistics.

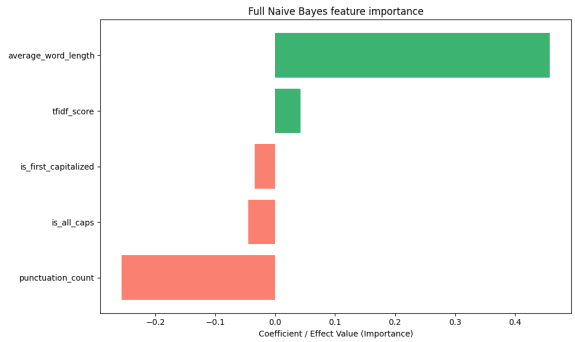

Figure 4: Feature importances for Naive Bayes (Statistical).

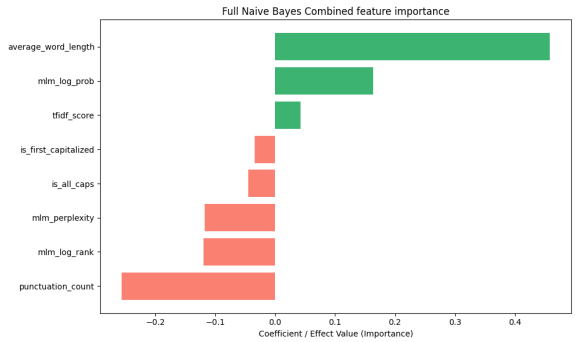

Figure 5: Feature importances for Naive Bayes (Combined).

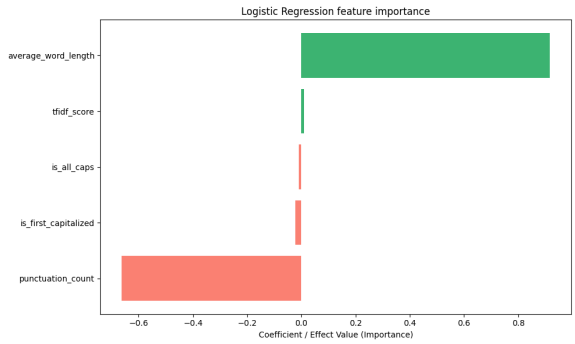

Figure 6: Feature importances for Logistic Regression (Statistical).

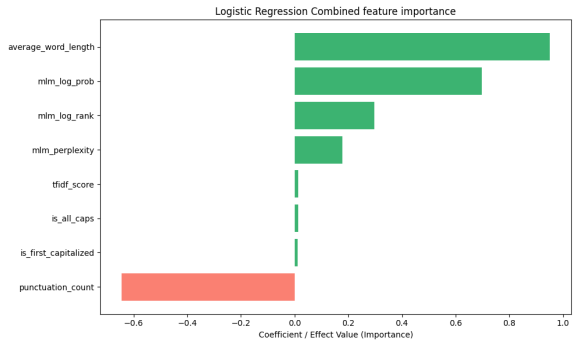

Figure 7: Feature importances for Logistic Regression (Combined).

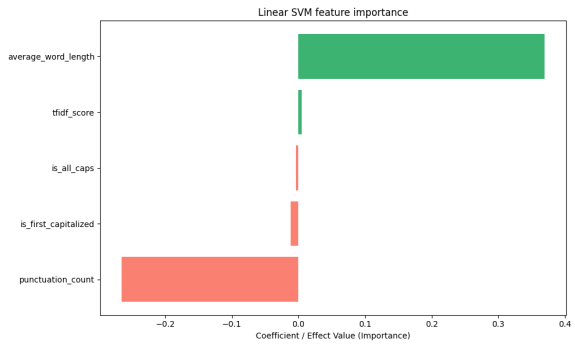

Figure 8: Feature importances for Linear SVM (Statistical).

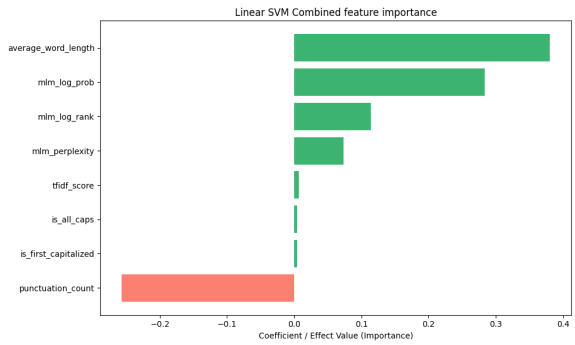

Figure 9: Feature importances for Linear SVM (Combined).

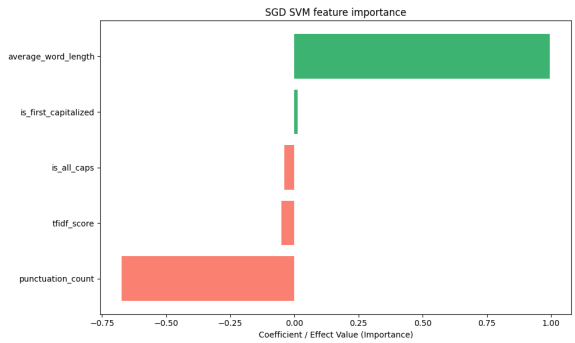

Figure 10: Feature importances for SGD SVM (Statistical).

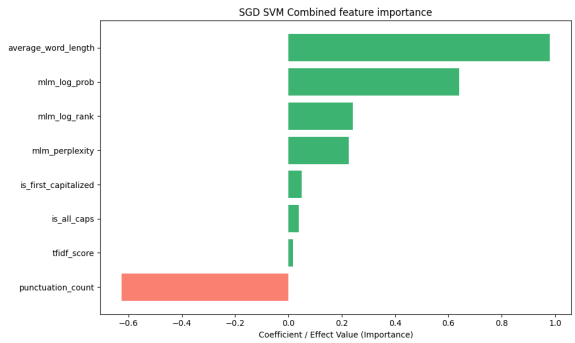

Figure 11: Feature importances for SGD SVM (Combined).

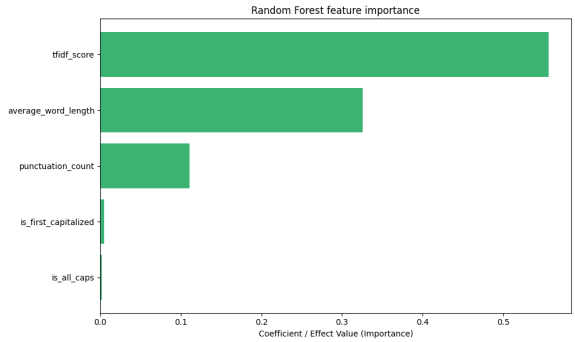

Figure 12: Feature importances for Random Forest (Statistical).

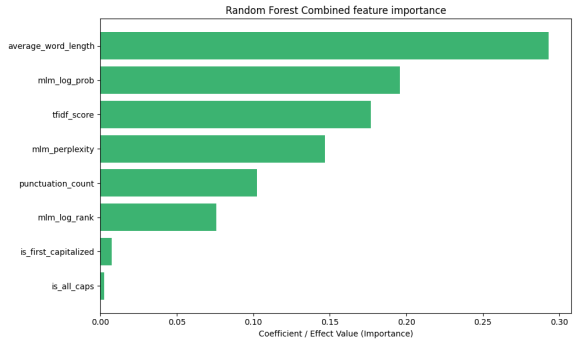

Figure 13: Feature importances for Random Forest (Combined).

C Individual Contributions

C.1 Anya Kiewit

C.2 Isabelle

C.3 Johan Wols

C.4 Marten de Jong

- Project Setup/Management
  - Trello
  - Python Project
  - Small bugfixes if occurred after merging
  - File-handling
- Preprocessing
  - Train-Val split
  - Context Windows
- Feature Engineering
  - MLM-features
- Evaluation
  - MAE
  - Visualization
- Models
  - SVM
- Report
  - MLM
  - SVM
  - Related work (partially)